DispatchPilot

A contract-aware AI dispatcher that plans the day, guarantees SLAs, and proves it.

Solution design and functional specification | Version 2 (post-review) | 11 November 2026

Summary. DispatchPilot is a field-operations platform for companies that maintain commercial buildings (elevators, AC, refrigeration). It reads repair requests and maintenance contracts, plans and re-plans each technician's day with an optimization engine, enforces service-level agreements, learns from completed work, and produces evidence of every response. We build it with React 19, Hono, Drizzle, PostgreSQL, Zod, and Better Auth, and we run it on free-tier services.

What changed in Version 2. In response to review feedback that the first design was too basic, this version adds a planning engine with minimal-disruption re-optimization, learned job durations, a simulator and benchmark, a contract and SLA engine with evidence packs, asset health with repeat-failure detection, a capacity what-if tool, and an AI quality layer (evaluation set and prompt-injection defenses). Section 1 lists them under "What makes DispatchPilot stand out."

Assignment requirement (Step 2)	Where we answer it
The role chosen and why	Section 1 (with differentiators, research, value proposition, and agent-versus-human comparison)
Key tasks and workflows the agent handles	Section 2 (with functional requirements)
High-level architecture, frontend and backend	Section 3 (with core engineering components and non-functional requirements)
Third-party APIs and integrations	Section 4
How the agent acts, and how users interact through the UI	Section 5
Supporting material	Appendix A: build plan. Appendix B: risks. Appendix C: references

1 The Role We Automate, and Why

Where this idea comes from. Nobody fixed the lift in our own office building, and the cause was not technical: it was a management problem. Repairs like this stall when nobody owns the request, nothing has a deadline, and no one follows up or escalates. A dispatcher agent closes exactly this gap. Every request gets an owner, a timer, and an escalation path, so nothing waits quietly until someone remembers it.

Role: field-service dispatcher | **Industry:** technical maintenance for commercial buildings (elevators, AC, refrigeration) | **Company profile:** about 8 technicians, 20 to 40 jobs a day, requests by email and chat, and maintenance contracts with response-time SLAs govern the work.

Today a human dispatcher reads requests, checks what the contract promises, chooses technicians, plans routes, keeps people informed, chases late jobs, and re-plans whenever something changes.

What makes DispatchPilot stand out

A dispatch tool that only scores technicians is easy to copy. The features below are what make this one hard to copy: product features that solve a buyer's real problem, and engineering features that prove the claims.

Product feature	What it does	Why it stands out
P1 Contract and SLA engine	Reads a maintenance contract PDF and turns priority tiers, response times, callback limits, and credits into rules and live timers. Produces a monthly evidence pack of timestamped responses.	The system enforces each promise and proves it. Customers can claim SLA credits only with proof of arrival [9].
P2 Asset registry and health	Every lift or AC unit has a service history. Repeated callbacks raise a risk score and a recommended inspection.	The lift story as a feature: the manager sees the risk before the outage.
P3 Capacity what-if	Replays a typical week in the simulator with different staffing (for example one more elevator technician) and shows projected SLA compliance and travel.	A decision tool for the owner, the clearest reason to pay.
P4 Plan review and explanation	Every re-plan shows a before-and-after diff with the reason, the SLA impact, and one-click approval.	Builds trust and keeps a human in control.
P5 Safety-aware triage	Safety language (for example a person trapped in a lift) takes the emergency path with the fastest escalation.	Protects people and matches contract response norms [9].

Engineering feature	What it does	Why it stands out
E1 Minimal-disruption planning engine	Optimizes travel, SLA-weighted lateness, overtime, balance, and a disruption penalty using insertion and local search, under hard constraints on skill, shift, and overlap.	A real algorithmic core, not a score formula.
E2 Learned durations with uncertainty	Estimates job time by equipment, technician, and building from completed jobs, as a median and a 90th percentile. SLA-critical jobs plan with the pessimistic value.	Plans get better as data arrives.
E3 Simulator and benchmark	A seeded digital twin of a working day compares the planner with a nearest-technician baseline, and runs inside automated tests.	Evidence instead of claims.
E4 Event-sourced core	Every state change becomes an event, so the team can replay a day and explain any decision. An outbox makes email sending reliable, and server-sent events keep the board live.	Audit-grade, reliable, and real time.
E5 Defended AI layer	Gemini acts only through typed, validated tools. An evaluation set measures extraction accuracy, and tests cover prompt-injection attacks in incoming emails.	We treat AI as production engineering.

Research: what the industry data shows

Figures below come from published field-service and facilities sources (numbered references in Appendix C). Software companies publish the sources marked *vendor*, so we treat them as directional and check them against our own measurements in the demo.

Finding	Figure	Ref.
Share of the technician day spent driving	2 to 4 hours a day, 25 to 50% of working time (vendor)	[4]
Travel per job in a field-service benchmark	86 minutes per job (Service Council, 2022)	[2]
Appointments that did not go according to schedule	47% (Salesforce survey of 350 US mobile workers, 2025)	[2]
First-time fix rate	Benchmark 70 to 75%; top teams above 80%	[1]
Healthy technician utilisation	Roughly 70 to 80% of working hours	[3]
Travel reduction from location-aware scheduling	15 to 25% less travel time per job (vendor)	[5]
Elevator contract response norms	30 to 60 minutes for entrapments; 4 hours for service calls	[9]
Public elevator contract example	60-minute response to unscheduled calls (City of Covina RFP)	[8]
Manual elevator callback dispatch	18 to 23 minutes from notification to dispatch (vendor)	[10]
SLA credits depend on proof	Customers recover credits only when they can prove the arrival time	[9]

Value proposition: would a company pay for this?

Who pays	What the problem costs	What DispatchPilot changes
Owner or operations manager of a maintenance company with 5 to 20 technicians. Building managers benefit through SLA evidence.	Drive time takes 25 to 50% of a technician's day [4]. Customers can claim SLA credits for late arrivals only with proof of the arrival time [9]. A human dispatcher costs about $47,000 to $49,000 a year in the US [6] and cannot watch every job around the clock.	Instant assignment, less travel, automatic delay handling, contract-aware SLA enforcement, and a timestamped record of every response. The dispatcher moves to approvals and exceptions.

Illustrative example (8 technicians). Technicians drive about 2.5 hours a day [4]. A 20% cut, inside the reported 15 to 25% range [5], frees 30 minutes per technician per day: 4 hours a day across the team, or about 1,000 hours a year (250 working days), roughly half of one full-time technician. We base this on planning assumptions, not guaranteed results.

Price anchor (2026 reported pricing)	Cost	Ref.
ServiceTitan-class platform (8 technicians)	$245 to $398 per technician per month: about $1,960 to $3,184 a month, plus $5,000 to $50,000 implementation	[7]
Mid-market tools (for example Housecall Pro)	$59 to $299 per user per month	[7]

These platforms digitise scheduling and dispatch boards. DispatchPilot's edge is an agent that plans under contract terms, explains its decisions, notifies people, escalates without waiting for a human to notice, and proves compliance.

Success metrics we will demonstrate

Metric	Baseline	Target
Time from request to assignment	18 to 23 minutes from notification to dispatch in manual elevator callbacks (vendor) [10]	Under 30 seconds to a proposed assignment
Technician travel time	86 minutes per job [2]; 15 to 25% cut reported for location-aware scheduling [5]	15 to 25% lower in the simulator against a nearest-free-technician baseline
SLA compliance	Nearest-technician baseline in the simulator	Higher compliance than the baseline across 100 simulated days
Plan stability after a delay	Manual re-plan moves many jobs	Moves the fewest jobs that keep SLAs safe, and reports the count per delay
Duration estimate accuracy	Flat default duration per job type	Lower average error than the default after 50 completed jobs
Contract and email extraction accuracy	None	90% or better on the evaluation set; the system handles every injection test safely
Jobs without an owner or deadline	Common when ownership is unclear (the lift story)	Zero: every job has an owner and an SLA timer

We set these targets as goals to measure with seeded data in the simulator, not as claims about real deployments.

Agent versus human dispatcher

Dimension	Human dispatcher	DispatchPilot
Speed on a new request	Minutes, depending on attention	Seconds; checks run every 5 minutes
Availability	Shift hours	Continuous, including nights and weekends
Planning under many constraints	Experience and rules of thumb	Optimizer weighing travel, SLA risk, balance, and disruption
Knowing what the contract says	Remembers or looks it up	Extracts rules once and enforces them on every job
Follow-up and escalation	Depends on memory	Automatic SLA timers and three-level escalation
Learning from history	Informal	Duration and asset-health models update from completed jobs
Unusual or safety-critical cases	Strong judgement	Defers to a human approval
Customer relationship and negotiation	Strong	Drafts messages only
Cost	About $47,000 to $49,000 a year in the US [6]	Free-tier services for the prototype

Verdict. For routine dispatch work (reading requests, matching, planning, notifying, chasing, re-planning) the agent does the job faster and without forgetting. A human stays in charge of approvals, exceptions, and safety decisions, so we hire the agent for coordination, not for judgement.

Scope

In scope (MVP)	Out of scope (roadmap)
Email and chat intake, contract ingestion and SLA rules, planning engine with re-optimization, SLA timers and escalation, duration learning, asset history and repeat-failure detection, simulator and what-if, requester notifications, role-based dashboard, approval queue, event log.	Invoicing and payments, parts inventory, native technician mobile app (status buttons simulate it), requester photo and voice intake, technician manual-search copilot, multiple companies sharing one deployment.

2 Key Tasks and Workflows

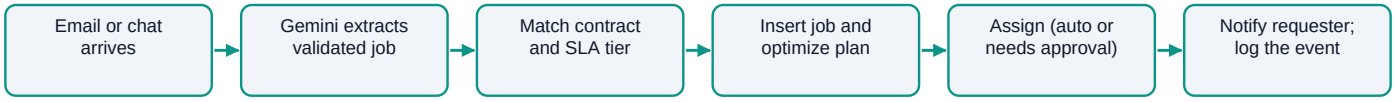
We break the role into sub-functions and rank them by impact. P1 items form the core loop, and we build them first.

#	Sub-function	What the agent does	Priority
1	Job intake and triage	Reads an email or chat message, extracts a validated job record, and routes safety-critical requests to the emergency path.	P1
2	Contract ingestion and SLA rules	Reads a maintenance contract and extracts response times, tiers, callback limits, and credits into rules a manager approves.	P1
3	Planning and assignment	Filters by hard constraints, then optimizes the day's plan and assigns the best technician with an explanation.	P1

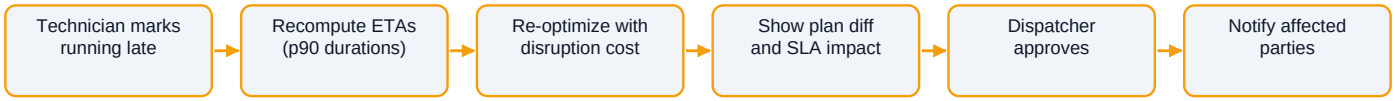
#	Sub-function	What the agent does	Priority
4	Delay handling and re-optimization	Detects late or unavailable technicians, re-plans with minimal disruption, and shows a before-and-after diff.	P1
5	SLA tracking and escalation	Gives every job a deadline from its contract tier and escalates stalled jobs step by step until someone acts.	P1
6	Simulation and benchmarking	Runs seeded working days to compare the planner with a baseline and to answer staffing what-if questions.	P1
7	Duration learning	Learns how long each kind of job takes by technician and building, as a median and a 90th percentile.	P2
8	Asset history and health	Records every visit per unit, detects repeat failures, and recommends inspections.	P2
9	Requester communication	Sends confirmation, ETA, progress, and delay notices by email.	P2
10	Calendar sync and weather planning	Creates technician calendar events and flags heat-wave demand surges.	P3
11	Reporting	Sends a morning plan, a weekly operations brief, and a monthly SLA evidence pack.	P2

Core workflows

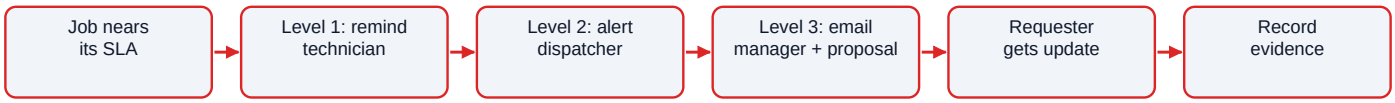
W1 New job (happy path)



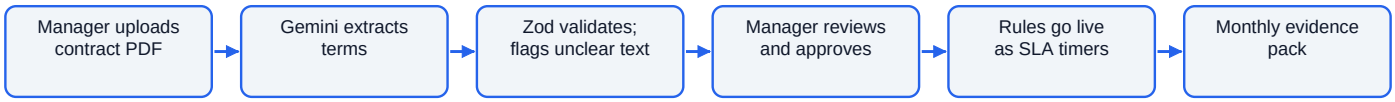
W2 Technician delay (the demo centrepiece)



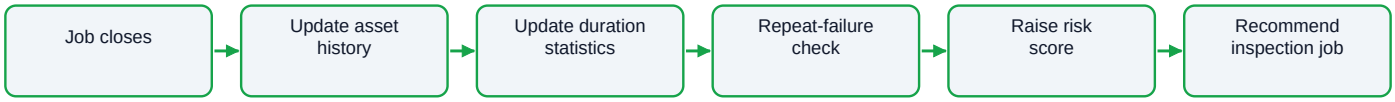
W3 SLA escalation (the lift problem, solved)



W4 Contract onboarding



W5 Learning and asset health



Workflow details

- **W1** is fully automatic for routine jobs. Low-confidence extractions and emergencies go to the approval queue first.
- **W2** is the demo moment: one click on a technician's status triggers a re-plan, and the diff shows which jobs moved and why.
- **W3** removes the failure behind the lift story: no job can wait silently. Escalation timing follows the contract tier, and safety-critical jobs escalate fastest.
- **W4 and W5** turn documents and history into rules and risk scores, which is what makes the product more than a scheduler.
- **Proactive checks** run on a schedule every few minutes: late technicians, jobs nearing their SLA, aging inbox items, and the daily forecast.

What the AI does versus what plain code does

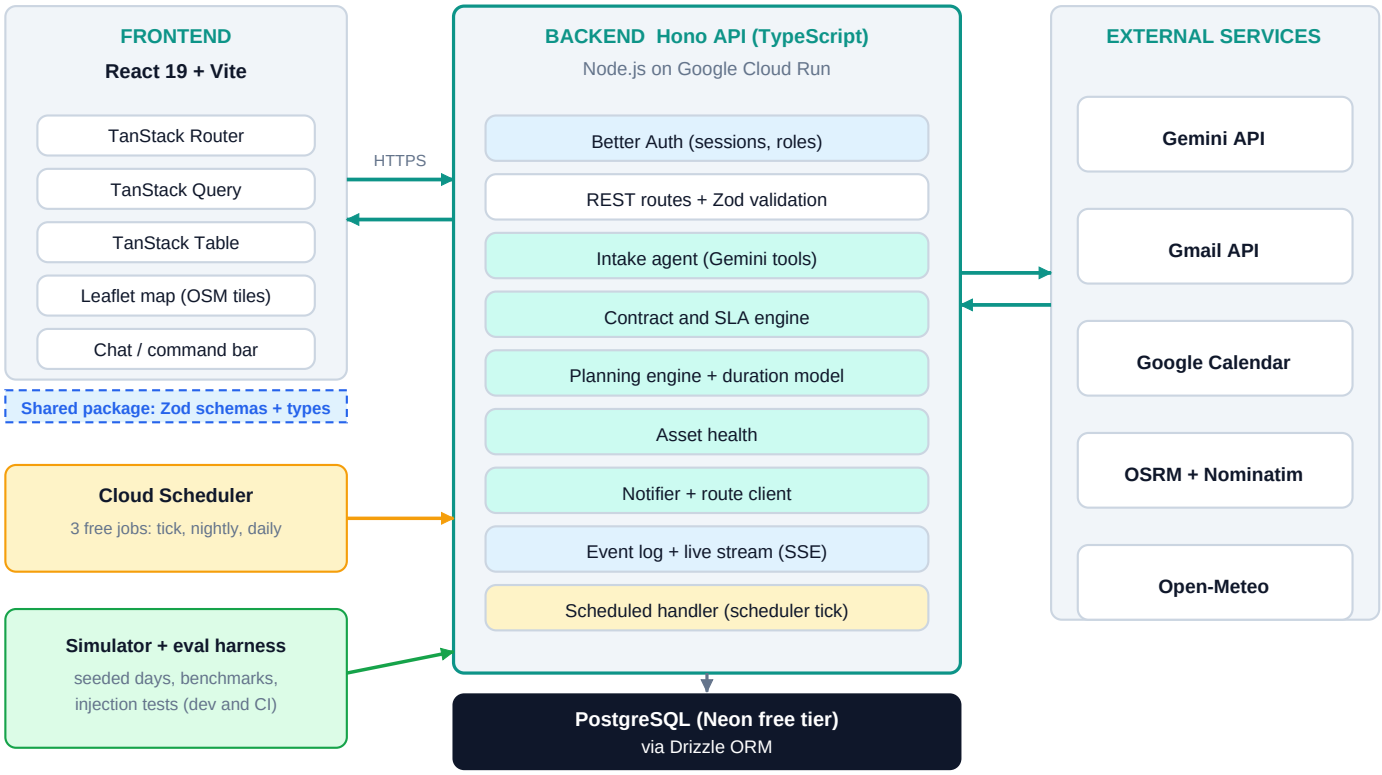
Gemini (language tasks)	Deterministic code (decisions and safety)
Understand messy emails; extract job fields; read contracts into draft rules; draft messages; explain plans in plain language.	Constraint filtering; optimization; time and distance maths; SLA timers; duration and asset statistics; idempotent actions; validation with Zod; event logging.

Functional requirements

ID	Requirement	Priority
FR-1	Ingest requests from email (Gmail) and a chat or command box, and create a job record.	P1
FR-2	Extract structured fields with Gemini, validate with Zod, and flag low-confidence results.	P1
FR-3	Detect safety-critical language and route the job to the emergency path.	P1
FR-4	Geocode addresses and cache the coordinates.	P1
FR-5	Let technicians update status (on my way, running late, done) and recompute ETAs.	P1
FR-6	Apply hard constraints: skill, shift, availability, and no overlapping jobs.	P1
FR-7	Optimize the plan with a cost covering travel, SLA-weighted lateness, overtime, balance, and disruption, using insertion and local search, within 2 seconds for 8 technicians and 40 jobs.	P1
FR-8	On a delay, re-optimize and show a before-and-after diff with reasons and SLA impact; apply on approval or when auto-pilot is on.	P1
FR-9	Estimate job durations from history (median and 90th percentile); use the 90th percentile for SLA-critical jobs.	P2
FR-10	Extract SLA terms from a contract PDF into structured rules, and activate them only after a manager approves.	P1
FR-11	Attach an SLA timer to every job from its contract tier and escalate in three levels (technician, dispatcher, manager).	P1
FR-12	Generate a monthly evidence pack of timestamped responses, with compliance and credit exposure on the dashboard.	P2
FR-13	Maintain an asset registry with the full service history of each unit.	P2
FR-14	Detect repeat failures (for example 3 or more callbacks in 60 days) and recommend an inspection job.	P2
FR-15	Run a seeded simulator of a working day and compare the planner with a nearest-technician baseline on SLA hit rate and travel.	P1
FR-16	Provide what-if analysis: change staffing and see projected SLA compliance and travel.	P2
FR-17	Maintain an evaluation set that measures extraction accuracy, plus prompt-injection test cases.	P2
FR-18	Provide role-based screens: dispatch board, jobs, plan review, contracts and SLA, assets, simulator, approvals and activity, my jobs, settings.	P1
FR-19	Record every state change as an event with its reasoning, and let users replay a day.	P1
FR-20	Update the board live through server-sent events, with polling as a fallback.	P2
FR-21	Email requesters a confirmation, ETA, delay notice, and completion message with a tracking link.	P2
FR-22	Run scheduled checks every 5 minutes; send a morning plan and a weekly operations brief.	P2
FR-23	Create and update technician calendar events; flag weather-driven demand surges.	P3

3 Architecture and Technical Approach

A typed full-stack TypeScript system. The LLM handles language; deterministic engines handle decisions.



Tech stack

Layer	Choice	Why
Frontend	React 19, TanStack Router / Query / Table, Leaflet	Typed routes, cached server state for live data, sortable tables, free map tiles.
Backend	Hono on Node.js, hosted on Google Cloud Run	Lightweight and typed; Cloud Run allows seconds of CPU for the planner and streaming for live updates, inside the Always Free tier named in the assignment.
Scheduling	Cloud Scheduler (3 free jobs)	One job for the 5-minute tick, one nightly refresh, and one daily digest and weekly brief.
Database	PostgreSQL (Neon) with Drizzle	Relational fit for jobs, contracts, assets, and events; typed queries and migrations.
Authentication	Better Auth with its Drizzle adapter	Self-hosted, TypeScript-first, with role-based access (see matrix below).
Validation	Zod on both sides	One schema validates API input, LLM output, and form data.
Planning	Custom TypeScript engine (insertion plus local search)	Runs inside the API process, testable, no paid solver.
AI	Gemini Flash-class model (AI Studio key)	Free tier, structured JSON output, and PDF input for contracts.

Core engineering components

Component	How it works	Why it is non-trivial
Planning engine	Builds routes by cheapest insertion, then improves them with relocate, swap, and 2-opt moves until a 2-second budget ends. Cost = travel + SLA-weighted lateness + overtime + imbalance + disruption (a penalty for moving a job we already promised).	Balances competing objectives under hard constraints, and re-plans with stability, not from scratch.
Duration model	The model computes median and 90th-percentile time per equipment type, technician, and building, and shrinks toward the equipment-type average when data is sparse.	Handles small samples and uncertainty.
SLA engine	The engine turns contract terms into rules (tier, response minutes, resolution target, callback cap, credit). Timers start at request time and drive the escalation ladder.	Turns unstructured documents into enforced behaviour.

Component	How it works	Why it is non-trivial
Asset health	The engine scores each asset by callback count, recency, and time to repair, and a threshold creates a recommended inspection job.	Predictive insight from operational data.
Simulator	A seeded generator creates jobs, breakdowns, and delays over a working day, runs policies side by side, and reports SLA hit rate, travel, and jobs moved.	Makes every claim measurable and repeatable in tests.
Event log	The system appends events with reasoning and derives state from them; an outbox sends each email exactly once; server-sent events push changes to the board.	Replayable, auditable, and reliable.
AI quality layer	Gemini calls typed tools; the system never treats email text as instructions; an evaluation set and injection tests run in CI.	Treats untrusted input as a security problem.

Roles and access matrix

Capability	Manager	Dispatcher	Technician	Requester
Sign-in	Account	Account	Account	None (private link by email)
View jobs and map	All	All	Own jobs	Own request
Approve or reassign jobs and plans	Yes	Yes	No	No
Update job status (on my way, late, done)	Yes	Yes	Own jobs	No
View activity log, escalations, and asset health	Yes	Yes	No	No
Manage contracts and SLA rules	Yes	View only	No	No
Run simulator and what-if	Yes	View only	No	No
Change autonomy settings (auto-pilot, dry-run)	Yes	No	No	No
Manage users, roster, and skills	Yes	View only	No	No

The backend enforces access on every API route, and the UI hides the pages a role cannot use.

Core data model

Table	Key fields
jobs	id, requester, address, lat/lng, equipment, asset_id, skill_required, urgency, safety_flag, status, confidence, contract_id, sla_due, escalation_level
technicians and skills	technician: id, user_id, name, shift, status, current_lat/lng, workload; skills: skill (elevator, refrigeration, chiller, split unit) and level per technician
assignments	job_id, technician_id, planned_start, eta, cost_breakdown (json), state (proposed / confirmed / reassigned)
contracts and sla_rules	contract: id, customer, source_pdf, status; rule: tier, response_minutes, resolution_target, callback_cap, credit_terms
assets and asset_events	asset: id, site, type, install_date, health_score; event: asset_id, job_id, outcome, time_to_repair
duration_stats	equipment_type, technician_id, building_id, n, median_minutes, p90_minutes
events	append-only: type, payload, reasoning, autonomy (auto / approved), created_at (the audit trail and the replay source)
simulation_runs	seed, policy, sla_hit_rate, travel_minutes, jobs_moved, created_at
Better Auth tables	user, session, account, verification, plus organization and member (with role)

Non-functional requirements

Category	Requirement
Security	Better Auth manages sessions; the server enforces role checks on every route; secrets live in environment variables; HTTPS only; requester links use unguessable tokens; the system treats incoming email and contract text as untrusted input.
Privacy	Demo runs on synthetic data only; store only the fields needed; send the LLM only what it needs.
Reliability	Idempotent actions, outbox for emails, retries with backoff, and a fallback for every integration; the system logs every failure and shows it to users.
Performance	A new job reaches a proposed assignment in under 30 seconds; the planner finishes in 2 seconds for 8 technicians and 40 jobs; main screens load in about 2 seconds on demo data.
AI quality	At least 90% extraction accuracy on the evaluation set, and the system handles every prompt-injection test case safely.
Usability	Responsive layout, one-click approvals, and plain-language explanations of every decision.
Maintainability	Shared Zod schemas, layered routes, services, and repositories, property tests that cover the planner, and types that run end to end.
Cost	Operate within the free tier of every service used.

4 Third-Party APIs and Integrations

Six free services, each behind an adapter with a fallback so the demo cannot break.

Service	Used for	Free-tier notes	Fallback if unavailable
Gemini API (Google AI Studio)	Extract jobs from messy emails; read contract PDFs into draft SLA rules; draft messages; explain plans in plain language.	Free tier, no card. AI Studio shows the per-model limits, so we use a Flash-class model and keep calls small.	Rule-based parser, manual rule entry, and message templates.
Gmail API	Read incoming job emails; send confirmations, ETAs, delay notices, and escalations.	Free. OAuth runs in test mode with our own account.	Simulated inbox endpoint for intake; logged outbound messages.
Google Calendar API	Create and update each technician's schedule events.	Free within standard quotas.	Internal timeline view only.
OSRM (OpenStreetMap data)	Drive times and distances for planning, ETAs, and route lines on the map.	The public demo server suits light use, so we cache results and batch requests.	Straight-line (haversine) distance with a speed factor.
Nominatim (OpenStreetMap)	Turn addresses into coordinates.	Free with a strict request rate, so we geocode once and store the result.	Pre-seeded coordinates.
Open-Meteo	Forecast heat waves and storms for demand surges and travel-delay risk.	Free, no API key for non-commercial use.	Skip the weather feature; the core loop keeps working.

We also use Leaflet with OpenStreetMap tiles for the map view.

Integration pattern

- **Adapter per service.** Each integration implements a small interface (for example *getTravelTime(a, b)*), so a real client and a fallback are interchangeable.
- **Cache and rate-limit.** We store geocoding and routing results in Postgres and queue outbound calls to respect free-tier limits.
- **Typed boundaries.** We parse every response from an external service or from Gemini with Zod before use.
- **Failure is visible.** If a service fails, the agent switches to its fallback, flags it in the event log, and continues.
- **Secrets.** API keys and OAuth tokens live in environment secrets, never in the repository.

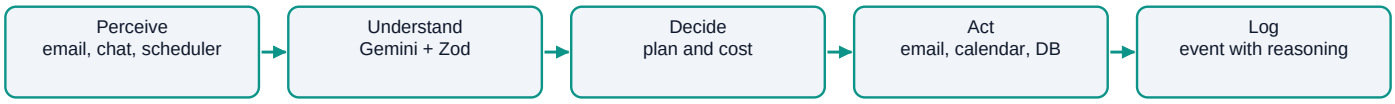
Example: one job through the integrations

Step	Service	Result
1	Gmail	Email arrives: "The lift on our floor has been out of service since this morning, please send someone today."
2	Gemini + Zod	Job: equipment = elevator, skill = elevator, urgency = high, safety-relevant, window = today.
3	SLA engine	Matches the customer's contract tier and sets the response deadline (for example 4 hours).
4	Nominatim + OSRM	Geocodes the address and computes drive times from each eligible technician's position.
5	Planning engine	Inserts the job, optimizes the day, and proposes the best technician with a cost explanation.
6	Gmail + Calendar	Requester gets a confirmation with ETA and a tracking link; the technician's calendar gets the visit.
7	Scheduler + Gmail	If the SLA nears without progress, the agent alerts the dispatcher and then the manager, and records the response as evidence.

5 How the Agent Automates Work and Takes Action

Perceive, understand, decide, act, log. Each step is explicit so the dispatcher can see and trust it.

Agent loop



How the agent decides (planning engine)

Hard constraints remove ineligible technicians first: required skill, within shift hours, no overlapping job, and reachable before the SLA deadline. The engine then minimises a weighted cost:

Cost component	Meaning	Effect on the plan
Travel time	OSRM drive time across the day's route	Favours shorter routes
SLA-weighted lateness	Predicted lateness against the contract deadline, using the 90th-percentile duration for critical jobs	Penalises lateness heavily
Overtime	Work beyond shift hours	Penalises overtime
Imbalance	Spread of workload across technicians	Favours an even spread
Disruption	Moves of jobs we already promised to a customer	Favours fewer moves

Each assignment stores its cost breakdown, and the UI shows a short explanation that Gemini writes.

Action catalogue and autonomy

Action	Trigger	Autonomy
Create job record	New email or chat message	Auto
Assign technician	Job triaged with high confidence	Auto (routine) / Approval (emergency or low confidence)
Activate SLA rules from a contract	Contract uploaded and parsed	Manager approval
Send confirmation and ETA	Assignment confirmed	Auto
Re-plan after a delay	Technician late or unavailable	Approval by default (plan diff); auto-pilot toggle available
Escalate a job nearing its SLA	Time-to-deadline threshold reached	Auto, three levels: technician, dispatcher, manager
Recommend an asset inspection	Repeat-failure threshold reached	Manager approval
Update duration and asset statistics	Job closed	Auto
Morning plan, weekly brief, monthly evidence pack	Scheduled job	Auto

Proactive behaviour (no human trigger)

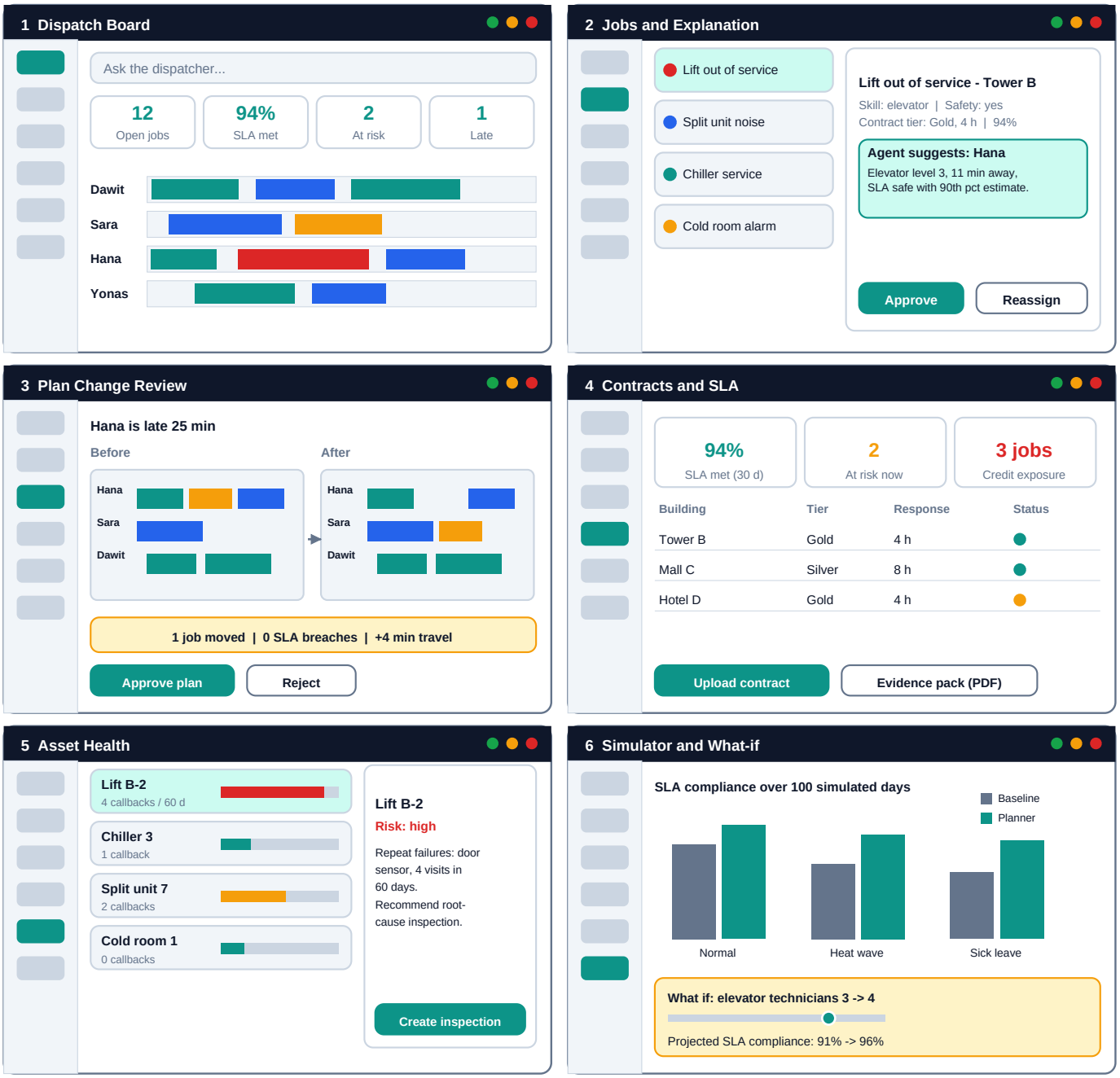
- **Every 5 minutes:** flag technicians behind schedule, escalate jobs nearing their SLA, and refresh ETAs.
- **Every night:** refresh duration statistics and asset health scores.
- **Every morning and week:** check the forecast, send the day's plan, and send the weekly operations brief with SLA compliance and risks.

User interaction and safeguards

- **Command bar:** natural-language requests such as "Technician Dawit is stuck in traffic, move his 2 pm job" or "What if we add one elevator technician?"
- **Approve, reject, or override:** the dispatcher can accept a plan, drag a job to another technician, or edit a message before the system sends it.
- **Auto-pilot toggle and confidence threshold** decide what runs without approval; only managers can change them.
- **Event log, idempotent actions, and replay** prevent duplicate emails and show what the agent did and why.
- **Untrusted input rule:** the agent treats text inside emails and contracts as data, never as instructions, and acts only through validated tools.
- **Dry-run mode** for the demo shows exactly what the agent would send without sending it.

User interface design

A single responsive web app: left navigation, a command bar on top, and one focused view at a time, with light and dark themes. What each user sees depends on their role. Status colours are consistent: teal = ok, blue = scheduled, amber = needs attention, red = late or urgent.



These wireframes are low-fidelity sketches; all numbers and names are placeholder sample data.

Pages and navigation

Page	Purpose	Main components	Roles
Sign in	Authenticate with Better Auth	Email and password, Google sign-in	All
Dispatch Board	Daily overview and re-planning	KPI cards, technician timelines, late-technician action	Manager, dispatcher
Jobs	Inbox, triage, explanation	Filterable table, cost explanation, approve or reassign	Manager, dispatcher
Plan Review	Approve re-plans	Before-and-after diff, SLA impact, approve or reject	Manager, dispatcher
Map	Live positions and routes	Leaflet map, route lines, ETA card, weather banner	Manager, dispatcher
Contracts and SLA	Rules, compliance, evidence	Contract upload, rule review, compliance and credit exposure, evidence pack	Manager (dispatcher view only)
Assets	Health and inspections	Asset list, risk score, history, create inspection job	Manager, dispatcher
Simulator	Benchmark and what-if	Policy comparison charts, staffing what-if	Manager (dispatcher view only)
Approvals and Activity	Human-in-the-loop and audit	Approval cards, event log with reasoning, day replay	Manager, dispatcher
My Jobs	Technician view	Own jobs, status buttons, navigation link	Technician
Settings	Roster, skills, autonomy	Skill editor, shift hours, auto-pilot toggle, dry-run switch	Manager

A Appendix A: Build Plan

A cut line holds the scope: Must items form the minimum submission, and we drop Could items first.

Priority	Items
Must	Authentication and roles; seeded data; email or chat intake with Gemini extraction and Zod validation; planning engine with hard constraints, cost function, and re-optimization with plan diff; SLA rules (from a contract, with manager approval), timers, and three-level escalation; simulator with baseline benchmark; event log; dispatch board, jobs, plan review, and contracts screens.
Should	Duration learning; asset history with repeat-failure detection; monthly evidence pack; what-if analysis; live updates by server-sent events; evaluation set with prompt-injection tests; auto-pilot toggle and dry-run mode.
Could	Live map with OSRM routes; Google Calendar events; weather surge alerts; weekly operations brief; requester tracking page.
Roadmap	Requester photo and voice intake; technician manual-search copilot; preventive-maintenance gap filling; multi-company tenancy.

Phase	Focus	Output
0 Foundations	Repository, schema, Better Auth, seed data, event log	Sign-in works, roles apply, and demo data and an events table exist
1 Core loop	Intake, extraction, assignment, confirmation	A job moves from email to an assigned technician
2 Planning engine	Constraints, cost function, optimizer, re-optimization, plan diff, property tests	A delay produces a minimal-disruption re-plan for review
3 SLA and contracts	Contract extraction, rule approval, timers, escalation, evidence pack	A contract becomes enforced SLAs with proof of response
4 Simulator and learning	Seeded simulator, benchmark, duration model, what-if	The planner beats the baseline with measured numbers
5 Assets and polish	Asset health, live updates, dashboards, evaluation set	The dashboard reads clearly and works for a demo
6 Documentation and handover	README and final documentation	A complete submission package

B Appendix B: Risks and Mitigations

Risk	Impact	Mitigation
Scope is large for the time available	Core loop unfinished	Must, Should, Could cut line; build planning and SLA engines before polish.
Planner is slow or produces poor plans	Weak core feature	Time-boxed search, property tests, and benchmark against the baseline from the start.
Hosting limits on CPU time or outbound calls	Planner or scheduled checks time out	The API runs on Cloud Run, which allows long CPU time. Edge-function free plans allow only about 10 ms of CPU per request, so we avoid them for the planner.
Contract extraction misreads a clause	Wrong SLA enforced	Zod validation, flags on unclear clauses, and manager approval before rules go live.
LLM extracts a job incorrectly, or email text manipulates it	Wrong assignment or action	Typed tools only, confidence threshold, approval queue, evaluation set, and injection tests.
Simulator results look unrealistic	Benchmark loses credibility	Document assumptions, publish seeds, and label results as simulated.
Free-tier limits change, or we exceed them	Features fail during the demo	Keep AI calls small, cache results, use fallbacks, and check current limits before building.
Gmail OAuth setup takes longer than expected	Intake or notifications blocked	Simulated inbox endpoint for intake; real Gmail only for sending, with logged fallback.
Regional restrictions on Gemini keys, GCP, or OAuth	Cannot create keys or projects	Contact the recruiter as the assignment advises; use alternative hosting.
Demo fails live	Weak first impression	Seeded data, dry-run mode, and a recorded fallback of the main scenario.

C Appendix C: References

We accessed these sources on 10 November 2026. Software vendors publish the items marked vendor, and we use them as directional evidence.

Ref.	Source
[1]	Arrivy, "How to Improve and Track Field Technician Productivity." arrivy.com/blog/how-to-improve-and-track-field-technician-productivity (vendor)
[2]	Stealth Agents, "Field Service Scheduling Workload Statistics 2026," summarising a 2022 Service Council benchmark and a 2025 Salesforce mobile-worker survey. stealthagents.com/research/field-service-scheduling-workload-statistics-2026
[3]	FieldEdge, "Technician Utilization Benchmarks." fieldedge.com/blog/technician-utilization-benchmarks (vendor)
[4]	Fieldproxy, "What Is Windshield Time and How Does It Impact Technician Productivity?" and "Scheduling Optimization to Reduce Windshield Time." fieldproxy.ai/articles (vendor)
[5]	Shifton, "The 12 Field Service KPIs That Actually Move the Needle." shifton.com/blog/field-service-kpis-2 (vendor)
[6]	US Bureau of Labor Statistics, OEWS May 2024, dispatchers except police, fire, and ambulance, as reported by Kaplan (median $49,030) and a BLS data mirror (median $46,620). Figures differ slightly between publishers.
[7]	NexaSphere, "What Field-Service Software Actually Costs in 2026"; CostBench, "Field Service Management Software Pricing 2026." nexasphere.io/blog, costbench.com/software/field-service-management
[8]	City of Covina, California, "RFP: City Wide Elevator Maintenance and Repair Services," 2024. covinaca.gov (public document)
[9]	OxMaint, "Facility Elevator and Escalator Maintenance Compliance." oxmaint.com/industries/facility-management (vendor)
[10]	Fieldproxy, "Elevator Emergency Callback Orders." fieldproxy.ai/blueprints/elevator-callback-orders (vendor)

Benchmarks come from mixed-quality sources. Before relying on any figure in a real deployment, validate it with the target company's own data.